A **Banner Grabber** is a useful cybersecurity tool that allows you to identify services running on a computer or server by connecting to open ports and retrieving information about the service.

For your cybersecurity scripting assignment, we'll build a beginner-friendly Python Banner Grabber that you can run in **Spyder 6**.

Assignment: Python Banner Grabber

1. What does a Banner Grabber do?

When a computer has an open port, a service is often listening for incoming connections. That service may reveal information when another computer connects to it.

A banner grabber attempts to collect information such as:

- Service name (FTP, SSH, HTTP, etc.).
- Software version.
- Server information.
- Operating system information (when disclosed).
- Other identifying information provided by the service.

Example:

If a banner grabber connects to port 22, it might return:

`SSH-2.0-OpenSSH_9.2`

This tells us that the system is running an SSH service using OpenSSH version 9.2.

2. Python Banner Grabber Script

Create a new Python file in Spyder 6 and name it:

`banner_grabber.py`

Copy the following code into your editor.

python

```python
import socket
import ssl

# ---------------------------------
# Banner Grabber
# ---------------------------------
```

```
print("=" * 45)
print("       CYBERSECURITY BANNER GRABBER")
print("=" * 45)

# Ask user for target computer
target = input("Enter IP address or hostname: ")

# Ports to scan
ports = [21, 22, 25, 80, 110, 143, 443, 445, 3306, 3389, 8080]

# Connection timeout
timeout = 2


# ---------------------------------
# Banner Grabbing Function
# ---------------------------------

def grab_banner(target, port):

    try:
        # Create TCP connection
        sock = socket.create_connection(
            (target, port), timeout=timeout
        )

        # Enable encrypted connection for HTTPS
        if port in [443, 8443, 9443]:

            context = ssl.create_default_context()
            context.check_hostname = False
            context.verify_mode = ssl.CERT_NONE

            sock = context.wrap_socket(
                sock, server_hostname=target
            )

        # Send HTTP request for web services
        if port in [80, 8080, 8000, 443, 8443, 9443]:

            request = (
                f"HEAD / HTTP/1.1\r\n"
                f"Host: {target}\r\n"
```

```
                f"Connection: close\r\n\r\n"
            )

            sock.sendall(request.encode())

        # Receive banner information
        banner = sock.recv(1024)

        # Close connection
        sock.close()

        if banner:
            return banner.decode(
                "utf-8", errors="replace"
            ).strip()

        else:
            return "Port is open, but no banner was returned."

    except socket.timeout:
        return "Connection timed out."

    except ConnectionRefusedError:
        return "Connection refused."

    except OSError as error:
        return f"Connection error: {error}"


# ---------------------------------
# Scan Target Ports
# ---------------------------------

print("\nTarget:", target)
print("\nStarting Banner Grab...\n")

for port in ports:

    try:

        # Check if port is open
        test_socket = socket.create_connection(
            (target, port), timeout=timeout
        )
```

```
            test_socket.close()

            print("-" * 45)
            print(f"Port {port}: OPEN")

            # Retrieve service banner
            banner = grab_banner(target, port)

            print("Banner Information:")
            print(banner)

        except (socket.timeout, ConnectionRefusedError):
            print(f"Port {port}: CLOSED or FILTERED")

        except OSError:
            print(f"Port {port}: Unable to connect")


print("\n" + "=" * 45)
print("           SCAN COMPLETE")
print("=" * 45)
```

---

3. Understanding the Code

Let's break down the important portions so you can explain how your program works.

Code	Purpose
`import socket`	Allows Python to establish network connections.
`import ssl`	Allows encrypted connections to HTTPS services.
`target = input()`	Asks the user which computer to scan.
`ports = [...]`	Contains the ports the program will examine.
`socket.create_connection()`	Attempts to establish a TCP connection.
`sock.recv(1024)`	Retrieves information sent by the connected service.
`banner.decode()`	Converts received network data into readable text.

Code	Purpose
`sock.close()`	Closes the network connection.
`try` and `except`	Handle errors without crashing the program.

4. Understanding the Ports

The script currently checks these commonly used ports:

Port	Service	Description
21	FTP	File Transfer Protocol
22	SSH	Secure remote connections
25	SMTP	Email transmission
80	HTTP	Web traffic
110	POP3	Email retrieval
143	IMAP	Email retrieval
443	HTTPS	Encrypted web traffic
445	SMB	Windows file sharing
3306	MySQL	Database service
3389	RDP	Windows Remote Desktop
8080	HTTP Alternate	Alternative web server port

---

5. Testing Your Program in Spyder 6

I recommend starting with your own computer rather than scanning an external system.

Test 1: Scan localhost

When prompted for the target, enter:

```text
```

```
127.0.0.1
```

This is your computer's loopback address.

The program will attempt to connect to each of the listed ports on your computer.

Test 2: Scan a computer on your network

You can also enter the local IP address of another computer that you own or have permission to test.

Test 3: Examine the results

You might see something like:

```text
-----------------------------------------
Port 22: OPEN
Banner Information:
SSH-2.0-OpenSSH_9.2

-----------------------------------------
Port 80: OPEN
Banner Information:
HTTP/1.1 200 OK
Server: Apache

-----------------------------------------
Port 443: CLOSED or FILTERED
```

These are illustrative results, not guaranteed outputs.

---

6. Important Cybersecurity Concepts

There are a few things worth understanding about banner grabbing:

Open port vs. banner information

An open port doesn't necessarily mean the service will provide identifying information. Some services don't send banners automatically.

Banner grabbing vs. port scanning

- Port scanning identifies which ports are open.
- Banner grabbing attempts to identify the services and software running behind those ports.

Why is this useful?

Banner grabbing helps cybersecurity professionals identify potentially outdated software, exposed services, and systems that might require security updates.

One limitation: software can hide or modify its banner information, so the results aren't always reliable indicators of the actual software version.

Security reminder: Only scan computers and networks you own or have explicit permission to assess.

---

7. Potential Improvements

Once you have the basic version working, there are several ways we can expand it:

1. **Custom Port Selection:** Allow users to enter their own ports instead of relying on a predefined list.
2. **Save Scan Results:** Export discovered banners into a `.txt` or `.csv` report.
3. **Service Identification:** Automatically associate discovered banners with known services.
4. **Improved Reporting:** Display open ports, service information, and potential security concerns in an organized report.
5. **Network Scanning:** Allow users to scan multiple authorized computers.

My recommendation: Start with the script above, get it running in Spyder 6, and then add the ability to save scan results. That would make it a more practical cybersecurity utility rather than just a program that displays information in the console.

Here were my results can you explain them to me?: Enter IP address or hostname: 127.0.0.1

Target: 127.0.0.1

Starting Banner Grab...

Port 21: CLOSED or FILTERED
Port 22: CLOSED or FILTERED
Port 25: CLOSED or FILTERED
Port 80: CLOSED or FILTERED
Port 110: CLOSED or FILTERED
Port 143: CLOSED or FILTERED
Port 443: CLOSED or FILTERED

Port 445: OPEN
Banner Information:
Connection timed out.

Port 3306: CLOSED or FILTERED
Port 3389: CLOSED or FILTERED
Port 8080: CLOSED or FILTERED

Absolutely! Your results are actually a good example of why banner grabbing is useful in cybersecurity. Let's break down what happened when you ran the script against your own computer (`127.0.0.1`).

1. Understanding Your Results

You entered:

```
text

127.0.0.1
```

This is known as the **localhost or loopback IP address**. It tells your computer to communicate with itself rather than another computer on your network.

Your script then attempted to connect to 11 commonly used network ports.

Here is what your results mean:

Port	Service	Result	Explanation
21	FTP	Closed/Filtered	No FTP service responded.
22	SSH	Closed/Filtered	No SSH service responded.
25	SMTP	Closed/Filtered	No SMTP service responded.
80	HTTP	Closed/Filtered	No HTTP web server responded.
110	POP3	Closed/Filtered	No POP3 email service responded.
143	IMAP	Closed/Filtered	No IMAP email service responded.
443	HTTPS	Closed/Filtered	No HTTPS web server responded.
445	SMB	**OPEN**	Windows file-sharing service accepted a connection.
3306	MySQL	Closed/Filtered	No MySQL service responded.
3389	RDP	Closed/Filtered	No Remote Desktop service responded.

Port	Service	Result	Explanation
8080	HTTP Alternate	Closed/Filtered	No alternate web server responded.

2. Why Is Port 445 Open?

This is probably the most interesting part of your results.

Port 445 is associated with SMB (Server Message Block).

SMB is a Windows networking protocol commonly used for:

- Sharing files between computers.
- Accessing shared folders.
- Sharing printers.
- Communicating with other Windows devices on a network.

Since you scanned your own computer, Windows is likely listening for SMB connections.

This is not necessarily a security problem. It is a common Windows networking service.

However, SMB is worth monitoring because improperly configured file sharing can expose sensitive information.

3. Why Did Banner Grabbing Time Out?

Your output showed:

```text
Port 445: OPEN

Banner Information:
Connection timed out.
```

This is an important distinction.

Your script actually performed two separate operations:

First: Port scanning

```python
test_socket = socket.create_connection(
    (target, port), timeout=timeout
)
```

This successfully connected to port 445.

That is why the program reported:

`Port 445: OPEN`

Second: Banner grabbing

```python
banner = sock.recv(1024)
```

This attempted to retrieve information from the service.

However, SMB doesn't necessarily send identifying information immediately after a connection is established.

Your script waited for the service to send something, but nothing arrived before the timeout.

Therefore, it reported:

`Connection timed out.`

This does NOT mean port 445 is broken or that your computer is experiencing a network problem.

It simply means your script didn't receive banner information using its current approach.

4. Why Are the Other Ports Closed or Filtered?

Most personal Windows computers don't run every network service.

For example:

- You probably aren't running an FTP server.
- You probably aren't hosting a website.
- You probably aren't running a MySQL database server.
- You may not have SSH enabled.

Your computer only needs to listen on ports associated with services that are running and configured to accept connections.

Also, your script groups closed ports and filtered ports together, so it cannot distinguish between the two.

5. What Could We Improve?

There is one improvement I would recommend making to your banner grabber.

Currently, your script checks whether a port is open and then attempts to retrieve a banner. However, different services communicate differently.

For example:

Service	Banner Grabbing Method
FTP	Usually sends a banner immediately.
SSH	Usually sends a banner immediately.
HTTP	Requires an HTTP request.
HTTPS	Requires an encrypted connection.
SMB	Requires an SMB protocol negotiation.

This means a basic `recv()` command won't work for every service.

For SMB specifically, we would need to implement an SMB negotiation request to retrieve protocol and server information.

---

6. Overall Assessment of Your Scan

Your results tell us:

1. Your computer has at least one accessible TCP service listening on the tested ports.
2. Port 445 is accepting connections.
3. Your computer did not return identifying banner information through the basic grabbing method.
4. The other ten tested ports did not accept connections.
5. There is no immediate indication of a security problem based solely on these results.

One thing worth remembering for your cybersecurity coursework:

A port being open doesn't automatically mean it's vulnerable. Likewise, a successful banner grab doesn't automatically mean the service is outdated or insecure.

Banner grabbing is an information-gathering technique used during the reconnaissance phase of a security assessment.

My recommendation: The next improvement should be teaching your script to recognize different services and use the appropriate banner-grabbing method for each one. That would make it considerably more effective as a cybersecurity tool.

So how can Improve the script, and teach me the steps

Absolutely. Since you're developing this in **Spyder 6 for your introductory cybersecurity scripting course**, I want to take you through this step by step rather than just give you a more advanced script.

Our goal is to turn your current program into a more reliable Banner Grabber that can:

1. Identify open and closed ports.
2. Recognize common network services.
3. Retrieve service banners when available.
4. Handle services that don't automatically send banner information.
5. Display results in an organized format.
6. Eventually save scan results to a report.

We'll build on the script you already have instead of starting from scratch.

Step 1: Understand what we're improving

Your previous scan produced:

```text
Port 445: OPEN
Banner Information:
Connection timed out.
```

The problem is that your original script successfully connected to the port but didn't know how to communicate with the SMB service.

Different network services communicate differently.

For example:

Service	How it responds
FTP	Usually sends a greeting automatically.
SSH	Usually sends a version banner automatically.
HTTP	Requires an HTTP request.
HTTPS	Requires an encrypted connection and HTTP request.
SMB	Requires an SMB protocol negotiation.
MySQL	Usually sends an initial binary handshake.

Our first improvement will be teaching the script to handle these differences.

Step 2: Update your Python script

Open your existing `banner_grabber.py` file in Spyder 6.

I recommend replacing your previous code with this updated version.

Improved Banner Grabber

python

```python
import socket
import ssl

# ----------------------------------------
# CYBERSECURITY BANNER GRABBER
# ----------------------------------------

print("=" * 50)
print("          CYBERSECURITY BANNER GRABBER")
print("=" * 50)

# Ask for target
target = input("\nEnter IP address or hostname: ")

# Ports to examine
ports = [21, 22, 25, 80, 110, 143, 443, 445,
         3306, 3389, 8080]

timeout = 3



# ----------------------------------------
# IDENTIFY SERVICES
# ----------------------------------------

services = {
    21: "FTP",
    22: "SSH",
    25: "SMTP",
    80: "HTTP",
    110: "POP3",
    143: "IMAP",
```

```
    443: "HTTPS",
    445: "SMB",
    3306: "MySQL",
    3389: "RDP",
    8080: "HTTP Alternate"
}


# ----------------------------------------
# BANNER GRABBING FUNCTION
# ----------------------------------------

def grab_banner(target, port):

    try:

        # Establish connection
        sock = socket.create_connection(
            (target, port), timeout=timeout
        )

    except ConnectionRefusedError:

        return "CLOSED", "Connection refused."

    except socket.timeout:

        return "TIMEOUT", "Connection attempt timed out."

    except OSError as error:

        return "ERROR", str(error)

    # Connection was successful
    try:

        sock.settimeout(timeout)

        # SMB requires protocol negotiation
        if port == 445:

            return "OPEN", (
                "SMB service detected by port number. "
                "Protocol negotiation is required "
```

```
            "to retrieve server information."
        )

    # HTTPS requires TLS encryption
    if port == 443:

        context = ssl.create_default_context()

        sock = context.wrap_socket(
            sock, server_hostname=target
        )

    # HTTP and HTTPS require a request
    if port in [80, 443, 8080]:

        request = (
            f"HEAD / HTTP/1.1\r\n"
            f"Host: {target}\r\n"
            f"Connection: close\r\n\r\n"
        )

        sock.sendall(request.encode())

    # Receive service information
    banner = sock.recv(1024)

    if banner:

        # MySQL may return binary data
        if port == 3306:

            return "OPEN", (
                "MySQL handshake data (hex): "
                + banner.hex()
            )

        return "OPEN", banner.decode(
            "utf-8", errors="replace"
        ).strip()

    else:

        return "OPEN", "No banner information received."
```

```
    except socket.timeout:

        return "OPEN", (
            "Connection successful, but the service "
            "did not return banner information."
        )

    except ssl.SSLError as error:

        return "OPEN", f"TLS negotiation error: {error}"

    except OSError as error:

        return "OPEN", f"Communication error: {error}"

    finally:

        sock.close()


# ----------------------------------------
# SCAN TARGET
# ----------------------------------------

print("\nTarget:", target)
print("\nStarting Banner Grab...\n")

for port in ports:

    service = services.get(port, "Unknown")

    status, banner = grab_banner(target, port)

    print("-" * 50)

    print(f"Port: {port}")
    print(f"Service: {service}")
    print(f"Status: {status}")

    if status == "OPEN":

        print("Banner Information:")
        print(banner)
```

```
    print()


# ----------------------------------------
# SCAN COMPLETE
# ----------------------------------------

print("=" * 50)
print("                 SCAN COMPLETE")
print("=" * 50)
```

Step 3: Learn what we changed

Let's go through the important improvements.

A. Service identification

Previously, your program only displayed the port number.

Now we have added:

python

```
services = {
    21: "FTP",
    22: "SSH",
    25: "SMTP",
    80: "HTTP",
    445: "SMB"
}
```

This is called a **dictionary** in Python.

A dictionary stores information using key-value pairs.

For example:

python

```
445: "SMB"
```

The key is 445, and the value is "SMB".

When the program encounters port 445, it can look up the corresponding service name.

This makes the output much easier to understand.

B. Improved connection handling

Previously, your program performed one connection to check whether the port was open and another connection to retrieve the banner.

We eliminated that duplication.

Now we establish the connection once:

```python
sock = socket.create_connection(
    (target, port), timeout=timeout
)
```

If the connection succeeds, we know the port accepted our TCP connection.

We can then attempt to retrieve its banner.

C. Improved HTTP detection

Your previous script sent HTTP requests, but we have made that process more organized.

```python
request = (
    f"HEAD / HTTP/1.1\r\n"
    f"Host: {target}\r\n"
    f"Connection: close\r\n\r\n"
)

sock.sendall(request.encode())
```

Let's break this down:

- `HEAD` asks the web server for response headers.
- `Host` identifies the website being requested.
- `Connection: close` asks the server to close the connection afterward.
- `sendall()` sends the request through the socket.

The server may respond with information identifying its web service.

D. SMB handling

This is particularly important based on your previous results.

```python
if port == 445:

    return "OPEN", (
        "SMB service detected by port number. "
        "Protocol negotiation is required "
        "to retrieve server information."
    )
```

Instead of waiting for SMB to send information that it normally doesn't send automatically, the program now explains why additional communication is necessary.

Important: This identifies SMB based on the conventional port number. It does not independently verify the service. Actual SMB identification requires protocol negotiation.

E. Improved error handling

We added several different error conditions.

Error	Meaning
`ConnectionRefusedError`	The connection was actively refused.
`socket.timeout`	The connection or response took too long.
`ssl.SSLError`	A problem occurred during TLS communication.
`OSError`	Another network-related error occurred.

This helps prevent the program from unexpectedly crashing.

Step 4: Run your updated script

Let's test it.

In Spyder:

1. Save your file.
2. Click the green Run button.
3. Wait for the program to ask for an IP address.
4. Enter:

```text
```

```
127.0.0.1
```

5. Press Enter.

Your results should look something like this:

```
text

==================================================
          CYBERSECURITY BANNER GRABBER
==================================================

Target: 127.0.0.1

Starting Banner Grab...

--------------------------------------------------
Port: 21
Service: FTP
Status: CLOSED

--------------------------------------------------
Port: 22
Service: SSH
Status: CLOSED

--------------------------------------------------
Port: 445
Service: SMB
Status: OPEN
Banner Information:
SMB service detected by port number.
Protocol negotiation is required to retrieve server information.

==================================================
                SCAN COMPLETE
==================================================
```

Your actual results will depend on which services are running on your computer.

Step 5: Understand what comes next

Now that we have a better foundation, there are three additional improvements I'd recommend.

Improvement	What you'll learn
SMB negotiation	How to communicate with Windows file-sharing services.
Automatic service detection	How to identify services based on their responses rather than just their port numbers.
Scan report generation	How to export results into a text file or CSV spreadsheet.

I would tackle these individually so you understand what each section of code accomplishes.

A little cybersecurity knowledge to take away

One important distinction:

Port scanning tells you what's accessible. Banner grabbing tells you what might be running behind those accessible ports.

For example, discovering that port 22 is open tells you SSH is potentially available. Retrieving a banner such as:

```text
SSH-2.0-OpenSSH_9.2
```

provides additional information about the software.

That information can help a cybersecurity professional determine whether further investigation or patching is necessary.